

Bhutan NDI

a deep tutorial — protocol, APIs, and four working integrations

This document walks through everything you need to integrate Bhutan’s National Digital Identity: the underlying SSI model, the cloud APIs you actually call, how events come back from the wallet, and four end-to-end applications — age verification, passwordless login, generic credential verification, and credential issuance — each with the wire format, decision logic, and example code.

Use the companion skill. A drop-in Claude skill that lets any AI coding agent build the integrations below from natural-language prompts is published at <https://github.com/sarangparikh22/bhutan-ndi-skill>. Clone it into `~/.claude/skills/` and ask your agent to “*add a Bhutan NDI age check to my Express app*” or “*why is my proof request returning 401?*” — the skill packs the protocol, payloads, edge cases, and pitfalls below into a form an agent can consult on demand.

Authored by **Sarang Parikh** <arangparikh22@gmail.com>

Contents

- What is Bhutan NDI?
- Architecture
- Step 1: Authenticate
- Step 2: The universal proof-request pattern
- Step 3: Receive events
- Application 1 — Age verification
- Webhook variant of age verification
- Application 2 — Passwordless login
- Application 3 — Generic credential verification
- Application 4 — Issue credential
- Build it with the skill

What is Bhutan NDI?

Bhutan NDI is a national-scale **self-sovereign identity** (SSI) deployment built on Hyperledger Indy / AnonCreds. Government agencies *issue* verifiable credentials (a citizenship VC, a permit, an address proof) directly into a citizen’s wallet on their phone. Apps like yours then ask that wallet — not the issuer — for cryptographic proof that specific attributes are true.

There are three roles in every flow:

Role	What they are
Issuer	A government agency that writes a credential to a wallet. <i>You're rarely this</i> — the issuer is a Bhutanese authority that's already onboarded.
Holder	A citizen with the Bhutan NDI Wallet on their phone, storing one or more VCs. <i>You never talk to them directly</i> ; the wallet acts on their behalf.
Verifier	You. You ask the wallet to prove an attribute (or several). The wallet generates a proof from the credential it holds and sends it back. You check the proof against the ledger's public keys — no roundtrip to the issuer needed.

Critically: **the verifier never talks to the issuer.** The wallet holds the credential itself; the proof is computed and presented in one step. That's what "self-sovereign" means — the holder controls their own data.

You almost always build verifier flows. To *issue* a VC into someone's wallet you need an on-ledger registration with the NDI Blockchain Team; we'll cover that later but it's not the common case.

Architecture

NDI publishes a hosted "cloud agent" so you don't have to run Indy infrastructure yourself. The surface is split across two hostnames:

Service	Host	What it does
Authentication	<code>staging.bhutanndi.com</code>	OAuth2 client_credentials → JWT (24h)
Verifier	<code>demo-client.bhutanndi.com</code>	Create proof requests, poll status
Issuer	<code>demo-client.bhutanndi.com</code>	Create relationships, issue credentials
Webhook control	<code>demo-client.bhutanndi.com/webhook</code>	Register/subscribe webhook callbacks
NATS event bus	<code>wss://natsdemoclient.bhutanndi.com</code>	Real-time async delivery of wallet events

The single biggest gotcha: auth lives on `staging.bhutanndi.com`, everything else on `demo-client.bhutanndi.com`. Hitting verifier endpoints on `staging.*` returns 401, not 404. The error doesn't tell you why.

Step 1: Authenticate

All API calls except `/authentication/v1/authenticate` require a bearer JWT. You mint one with OAuth2 `client_credentials`.

Request

```
curl -X POST 'https://staging.bhutanndi.com/authentication/v1/authenticate' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data-urlencode 'client_id=3tq7ho23g5risn90a76jre5f' \
  --data-urlencode 'client_secret=111rvn964mucumr6c3qq3n2poilvq5v92bkjh58p121nmoverquh' \
  --data-urlencode 'grant_type=client_credentials'
```

Response

```
{
  "access_token": "eyJraWQiOi...(JWT)",
  "expires_in": 86400,
  "token_type": "Bearer"
}
```

Cache it. The token is valid for 24 hours. Re-minting per request is wasteful and hits rate limits. A clean client caches it in process memory and refreshes five minutes before expiry to avoid edge races.

```
async _fetchToken() {
  const body = new URLSearchParams({
    client_id: this.clientId,
    client_secret: this.clientSecret,
    grant_type: "client_credentials",
  });
  const r = await fetch(`${this.authBase}/authentication/v1/authenticate`, {
    method: "POST",
    headers: { "Content-Type": "application/x-www-form-urlencoded" },
    body,
  });
  if (!r.ok) throw new Error(`NDI auth failed: ${r.status} ${await r.text()}`);
  const j = await r.json();
  this._token = j.access_token;
  this._tokenExpiresAt = Date.now() + (j.expires_in - 300) * 1000; // refresh 5min early
  return this._token;
}
```

```

}
async token() {
  if (!this._token || Date.now() >= this._tokenExpiresAt) await this._fetchToken();
  return this._token;
}

```

Step 2: The universal proof-request pattern

One endpoint — `POST /verifier/v1/proof-request` — powers three of our four applications. Only the `proofAttributes` list and the `purpose` field differ. Understand this once and the rest is variation.

Request

```

POST /verifier/v1/proof-request
Host: demo-client.bhutanndi.com
Authorization: Bearer <JWT>
Content-Type: application/json

```

```

{
  "proofName": "human-readable label shown in the wallet UI",
  "proofAttributes": [
    {
      "name": "Date of Birth",
      "restrictions": [
        { "schema_name": "https://dev-schema.ngotag.com/schemas/c7952a0a-..." }
      ]
    }
  ],
  "purpose": "ekyc", // or "login", "ekyc_update"
  "authenticationLevel": "Standard",
  "isShortenUrl": true
}

```

The fields that matter:

- `proofAttributes[].name` — the exact attribute name from the schema. Case-sensitive.
- `proofAttributes[].restrictions[].schema_name` — pins the proof to one schema. Without this the wallet would accept self-attested data.
- `purpose` — must be `"login"`, `"ekyc"`, or `"ekyc_update"`. The wallet uses this to label what it's about to share.
- `authenticationLevel` — `"Standard"` is what staging uses. Higher levels exist for production (biometric, OTP-confirmed).

Response

```
{
  "statusCode": 201,
  "message": "Proof URL created successfully",
  "data": {
    "proofRequestThreadId": "50cc681c-ccc1-4eae-92de-b7724f6fad18",
    "deepLinkURL": "bhutanndidemo://data?url=https://stage-demo-shortening-url.s3...amazonaws.com/default/..."
  }
}
```

You now have three pieces of state to keep in mind:

- **proofRequestThreadId** — your correlation key. The wallet's eventual response will arrive tagged with this ID. Store it server-side so you can route the answer back to the right user/session.
- **proofRequestURL** — encode this into a QR code for desktop users.
- **deepLinkURL** — use this as a tap-link on mobile (or any context where the wallet is on the same device as the browser).

Rendering the QR + deep link

```
// Wrap any proof-request response with a QR data URL.
import QRCode from "qrcode";

async function withQR(data) {
  const qrSvg = await QRCode.toString(data.proofRequestURL, {
    type: "svg", margin: 0,
    color: { dark: "#0b1220", light: "#ffffff" },
  });
  return { ...data, qrSvg };
}
```

The deep-link scheme is `bhutanndidemo://` on staging and `bhutanndi://` in production. Don't hardcode the scheme; render whatever the server returns verbatim.

What happens next

The wallet shows the user what's being requested. If they approve, it generates a cryptographic proof from the credential it holds, and POSTs that proof to the NDI cloud agent. Then the result is delivered to *you* asynchronously — that's what Step 3 is about.

What about GET /verifier/v1/proof-request?threadId=...?

It exists, and it tells you the *status* of a request (`proofInvitationCreated`, etc.) — but **it does not return the revealed attributes**. Those

only come over NATS or webhook. Don't try to poll your way to the proof; you can't.

Step 3: Receive events

The wallet doesn't reply directly to your POST `/verifier/v1/proof-request`. Instead, NDI's cloud agent receives the wallet's proof and pushes it to you over one of two transports.

Transport A — NATS

Your server opens an outbound WebSocket Secure connection to NDI's NATS server. NDI publishes the proof result on a subject named after the `threadId`, and your subscriber receives it. Best when you can keep a long-lived outbound connection open (k8s, EC2, dedicated host) and you don't want to host a public HTTPS endpoint.

```
import * as nats from "nats";

const nc = await nats.connect({
  servers: ["wss://natsdemoclient.bhutanndi.com"],
  authenticator: nats.nkeyAuthenticator(
    new TextEncoder().encode("SUAPXY7TJFUFE3IX3OEMSLE3JFZJ3FZZRSRSOGSG2ANDIFN7702MIBHWUM")
  ),
});

// NDI publishes the SAME proof result on four subjects simultaneously:
// <bare threadId>
// VERIFIER_SERVICE/finalProofReqStatusUpdate
// VERIFIER_SERVICE/proofReqStatusUpdate
// webhook/webhook-common-pattern
// We subscribe wildcard ">" and route by the `pattern` field.
const sub = nc.subscribe(">");
for await (const msg of sub) {
  const wrap = JSON.parse(msg.string());
  const threadId = wrap.pattern || wrap.data?.thid;
  if (threadId && activeThreads.has(threadId)) handle(threadId, wrap);
}
```

Transport B — Webhook

NDI POSTs the proof result to a public HTTPS endpoint you host. NDI calls your OAuth2 token endpoint first to mint a bearer, then attaches it to the webhook delivery. Best for serverless platforms, production with audit/retry requirements, or organizations that prefer inbound HTTPS to outbound WSS.

```

// Two endpoints you have to host.
app.post("/oauth/token", (req, res) => {
  const { client_id, client_secret } = req.body || {};
  if (client_id !== "my-app" || client_secret !== MY_SECRET) {
    return res.status(401).json({ error: "invalid_client" });
  }
  res.json({ access_token: MY_SECRET, expires_in: 3600, token_type: "Bearer" });
});

app.post("/ndi-webhook", (req, res) => {
  if (req.headers.authorization !== `Bearer ${MY_SECRET}`) return res.sendStatus(401);
  const body = req.body;
  const threadId = body.pattern || body.data?.thid;
  if (threadId) routePayload(threadId, body, "webhook");
  res.json({ ok: true });
});

```

Register the webhook once, then subscribe each new `threadId` as you create proof requests:

```

# once per app deploy
curl -X POST 'https://demo-client.bhutanndi.com/webhook/v1/register' \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{
  "webhookId": "my-app",
  "webhookURL": "https://my-public-host/ndi-webhook",
  "authentication": {
    "type": "OAuth2", "version": "v1",
    "data": {
      "url": "https://my-public-host/oauth/token",
      "grant_type": "client_credentials",
      "client_id": "my-app",
      "client_secret": "<a secret you choose>"
    }
  }
}'

# per threadId
curl -X POST 'https://demo-client.bhutanndi.com/webhook/v1/subscribe' \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{ "webhookId": "my-app", "threadId": "<your threadId>" }'

```

For local development, `ngrok http 3030` gives you a public URL.

The shared payload envelope

Both transports deliver the same JSON structure — a NestJS-style pub-sub envelope:

```
{
  "pattern": "<threadId>",
  "data": {
    "type": "present-proof/presentation-result",
    "verification_result": "ProofValidated",
    "requested_presentation": {
      "revealed_attrs": {
        "Date of Birth": [{ "value": "19/07/1995", "identifier_index": 0 }]
      },
      "unrevealed_attrs": {},
      "predicates": {},
      "self_attested_attrs": {},
      "identifiers": [
        { "schema_id": "https://dev-schema.ngotag.com/schemas/c7952a0a-...",
          "cred_def_id": null }
      ]
    },
    "relationship_did": "82945247-970f-43a5-9da9-e8a52fb530e0",
    "thid": "<threadId>",
    "holder_did": "did:key:z6MkjKxng36Cy4KRtn2Gsn7a3kEHIP9JhGYw72Tu5qzj85jG"
  }
}
```

Three normalization rules — both transports need them:

1. **Unwrap data.** The real payload is always nested.
2. **revealed_attrs[name] can be an array or an object.** When multiple credentials match, it's an array. Always: `Array.isArray(v) ? v[0]?.value : v?.value`.
3. **Trust verification_result === "ProofValidated", not the presence of revealed_attrs.** A failed proof still has the structure populated.

The router

Because both transports converge on the same envelope, your routing function is transport-agnostic. A flow can be tagged with which transport it expects; that lets you build a clean side-by-side demo that proves both work.

```
function routePayload(threadId, payload, via /* "nats" | "webhook" */) {
  const inner = payload?.data?.type ? payload.data : payload;
  const flow = flows.get(threadId);
  if (!flow) return;
  if (flow.transport && flow.transport !== via) return;  // wrong channel for this flow
}
```



```

if (flow.result || flow.relationship) return; // dedup

if (inner.type === "present-proof/presentation-result" || inner.requested_presentation) {
  handleProofResult(threadId, inner, via);
} else if (inner.relationship_id || inner.relationshipDid) {
  handleRelationshipResult(threadId, inner, via);
}
}

```

Application 1 — Age verification

Ask the wallet for Date of Birth, compute age server-side, return 18+ / under 18. This is the canonical proof-request flow with the absolute minimum disclosure: one attribute.

The request

```

app.post("/api/age/start", async (_req, res, next) => {
  try {
    const data = await ndi.createProofRequest({
      proofName: "Age verification (18+)",
      attributes: ["Date of Birth"],
      purpose: "ekyc",
    });
    watchThread(data.proofRequestThreadId, "age", "nats");
    res.json(await withQR(data));
  } catch (e) { next(e); }
});

```

The decision

Date of Birth is delivered as the string "DD/MM/YYYY". That's Bhutan's conventional format, not ISO — parse explicitly to avoid month/day confusion. Then compute age with the standard “before-birthday” adjustment:

```

export function parseDOB(dmy) {
  const [d, m, y] = dmy.split("/").map(Number);
  return new Date(Date.UTC(y, m - 1, d));
}

export function ageOn(dob, ref = new Date()) {
  let age = ref.getUTCFullYear() - dob.getUTCFullYear();
  const beforeBirthday =
    ref.getUTCMonth() < dob.getUTCMonth() ||
    (ref.getUTCMonth() === dob.getUTCMonth() && ref.getUTCDate() < dob.getUTCDate());
  if (beforeBirthday) age -= 1;
}

```

```

    return age;
}

// In the proof handler:
const dobStr = pickValue(payload.requested_presentation.revealed_attrs["Date of Birth"]);
const age = ageOn(parseDOB(dobStr));
const isAdult = payload.verification_result === "ProofValidated" && age >= 18;

```

Privacy footnote

This reveals the exact DOB to your back-end. If you only need yes/no, parse it in memory, store the boolean, and drop the string before persisting. A true zero-knowledge age predicate (prove “18” without revealing the date) is technically possible in AnonCreds but requires a schema and credential definition designed for predicates — the demo Foundational ID schema doesn’t currently expose age as a numeric attribute, so we use the DOB string. Coordinate with the NDI Blockchain Team if you need ZK predicates in production.

Webhook variant of age verification

Identical proof-request, but the result comes back through the webhook endpoint instead of NATS. Useful to prove the two transports really are equivalent.

What changes server-side

```

app.post("/api/age-webhook/start", async (_req, res, next) => {
  if (!webhookId) {
    return res.status(503).json({ error: "Webhook transport not available" });
  }
  const data = await ndi.createProofRequest({
    proofName: "Age verification (webhook)",
    attributes: ["Date of Birth"],
    purpose: "ekyc",
  });
  watchThread(data.proofRequestThreadId, "age-webhook", "webhook");
  await ndi.subscribeWebhook(webhookId, data.proofRequestThreadId); // ← the new line
  res.json(await withQR(data));
});

```

Two extra things had to happen *before* this route can work, both at server startup:

1. **ngrok auto-spawn.** The server runs `ngrok http 3030`, then reads the public URL from ngrok’s local API at `http://127.0.0.1:4040/api/tunnels`.
2. **Register a webhook with NDI** using that public URL plus an OAuth2 config pointing at our own `/oauth/token` endpoint.

Application 2 — Passwordless login

A login flow is the same proof-request endpoint with a different attribute list and `purpose: "login"`. The wallet shows the user “App X is asking you to sign in” rather than “App X needs to verify your identity.”

The request

```
app.post("/api/login/start", async (_req, res, next) => {
  const data = await ndi.createProofRequest({
    proofName: "Sign in with NDI",
    attributes: ["Full Name", "ID Number"],
    purpose: "login", // ← that's the whole login twist
  });
  watchThread(data.proofRequestThreadId, "login", "nats");
  res.json(await withQR(data));
});
```

Mapping the proof to a session

ID Number is the field a human recognizes, but `holder_did` is the right *primary key*:

- `holder_did` is a `did:key`: identifier derived from a keypair that lives in the wallet. It's stable per wallet install across credential re-issuances.
- ID Number is human-meaningful but can theoretically change (re-issuance after correction). Show it; don't key off it.

```
function handleProofResult(threadId, payload, via) {
  if (payload.verification_result !== "ProofValidated") return reject();
  const r = payload.requested_presentation.revealed_attrs;
  const get = (k) => Array.isArray(r[k]) ? r[k][0].value : r[k]?.value;
  const session = {
    holderDID: payload.holder_did, // stable primary key
    idNumber: get("ID Number"),
    fullName: get("Full Name"),
    via,
  };
  saveSession(threadId, session);
  // Issue a session cookie / JWT back to the browser at this point.
}
```

Edge cases worth thinking about

- **Wallet reinstall.** A new wallet means a new keypair means a new `holder_did`. Have an account-recovery path that re-links to an existing

account via ID Number after a step-up check.

- **Replay.** The proof is bound to `threadId` and to the cryptographic nonce in the NDI request. You don't need extra anti-replay machinery as long as you only finalize a session once per `threadId`.
- **Session length.** The proof tells you the user authenticated *now*. Re-auth on a schedule by issuing a fresh proof request, or trust your cookie for as long as you'd trust any other session.

Application 3 — Generic credential verification

“Verify credential” is the same proof-request endpoint with a caller-defined attribute list. The most useful capability hidden inside it: **one proof request can pull attributes from multiple credentials**. Mention a different `schema_name` per attribute and the wallet composes sub-proofs from each underlying VC.

The request

```
app.post("/api/verify/start", async (req, res, next) => {
  const attrs = Array.isArray(req.body?.attributes) && req.body.attributes.length
    ? req.body.attributes
    : ["Full Name", "ID Number", "Dzongkhag", "Gewog"];
  const data = await ndi.createProofRequest({
    proofName: req.body?.proofName || "Verify Foundational ID",
    attributes: attrs,
    purpose: "ekyc",
  });
  watchThread(data.proofRequestThreadId, "verify", "nats");
  res.json(await withQR(data));
});
```

Multi-credential proof shape

```
"proofAttributes": [
  { "name": "Full Name",
    "restrictions": [{ "schema_name": "https://.../foundational-id" }] },
  { "name": "House Number",
    "restrictions": [{ "schema_name": "https://.../permanent-address" }] },
  { "name": "Permit Number",
    "restrictions": [{ "schema_name": "https://.../work-permit" }] }
]
```

The response then has all three keys in `revealed_attrs`, plus three entries in `identifiers[]` — one per source credential. The `identifier_index` field on each revealed attribute tells you which credential it came from. That's how

you'd implement “Verify Foundational ID *and* a current Work Permit issued by issuer X” in a single round-trip.

Restricting by issuer

Add an `issuer_did` to the restriction:

```
"restrictions": [{
  "schema_name": "https://.../foundational-id",
  "issuer_did": "did:indy:bhutan:Foo..."
}]
```

Application 4 — Issue credential

Issuance is fundamentally different from the other three flows. You're not verifying something the wallet already has — you're writing something new into it. The NDI cloud agent auto-resolves the on-ledger `credDef` for your tenant from the `schemaId` you pass, so you don't deal with `credDefIds` in the request body. Two practical steps:

1. **Discover the holder's DID.** NDI needs a `holderDID` (`did:key:...`) to address the credential offer. The most reliable way: run a tiny proof-request for any attribute, then read `holder_did` off the response.
2. **Issue the credential.** One POST with `schemaId` (web URL form), `holderDID`, and a `credentialData` object whose keys + types match the schema exactly.

Step 1 — discover the holder DID

```
app.post("/api/issue/discover-holder", async (_req, res, next) => {
  const data = await ndi.createProofRequest({
    proofName: "Confirm identity to receive a credential",
    attributes: ["Full Name"],
    purpose: "ekyc",
  });
  watchThread(data.proofRequestThreadId, "issue-discover", "nats");
  res.json(await withQR(data));
});
```

The wallet sees “share your Full Name”, taps approve, and the NATS / webhook event arrives with `holder_did`. You ignore `Full Name` if you don't need it; you only care about the DID.

Step 2 — issue

```
curl -X POST 'https://demo-client.bhutanndi.com/issuer/v1/issue-credential' \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
```

```
-d '{
  "credentialData": {
    "Issuer Name": "Demo",
    "Student ID": 1234,
    "Student Name": "Dorji",
    "Title of Award": "B.Sc. Computer Science",
    "College Name": "Demo College"
  },
  "schemaId": "https://dev-schema.ngotag.com/schemas/ff021513-94b1-407d-a0ee-bb829531df42",
  "holderDID": "did:key:z6Mkxh1pxrNMc4gNv9srXY2pHFxED8cJCuSV3UdgKiybp44Z"
}'
```

Response:

```
{
  "statusCode": 201,
  "message": "Credential offer created successfully",
  "data": {
    "credInviteURL": "https://stage-demo-shortening-url.s3...",
    "deepLinkURL": "bhutanndidemo://data?url=https://stage-demo-shortening-url.s3...",
    "issueCredThreadId": "e49f8880-99c7-4f1e-b6cd-9de2c0ba83da",
    "relationshipDid": "7d1dbade-5c88-4681-bf08-5a77bf53d570",
    "revocationId": "304e436e-acb3-4410-b0fa-424eb81d0343"
  }
}
```

Render the `credInviteURL` as a QR and have the holder scan it. They'll see a "do you want to accept this credential?" prompt; once they tap accept, the credential lands in their wallet. Track `issueCredThreadId` on NATS/webhook for the acceptance event.

Type-matching: a real footgun

The endpoint validates each attribute against the schema's declared type. The Academic Award schema above expects `Student ID` as a **number**; passing "1234" (string) returns:

```
{
  "statusCode": 400,
  "message": "Validation failed: Attribute Student ID has type string but expected type number"
}
```

Keep type metadata next to each attribute in your client and coerce at the boundary. Don't assume strings.

Which schemas can you issue?

NDI maintains a public catalog of endorsed schemas in this sheet: https://docs.google.com/spreadsheets/d/1jLKahG_mTGIZr2MNm1hUka3GQ1vu3QOa2X

dK5gv6Eio/edit?gid=0

Anyone can **verify** any of these — issuance, however, requires NDI to have registered a credDef under your tenant for the schema. Email your integration contact saying “*please register a credDef for `client_id` <yours> against the <schemaName> schema*” and they’ll set it up. Once it’s registered, the same POST `/issuer/v1/issue-credential` you’d already use just starts succeeding for that schemaId — no client-side change.

Calling the endpoint with a schemaId that hasn’t been registered for your tenant returns “Schema details not found” (404).

Build it with the skill

Everything above — the host split, the proof-request DTO, the NATS subjects, the envelope normalization, the DOB parsing edge cases, the credDefId workflow — is packaged as a Claude skill at:

<https://github.com/sarangparikh22/bhutan-ndi-skill>

Drop it into `~/.claude/skills/` (or install it as a Claude Code plugin) and any Claude-powered agent can build all four applications from natural-language prompts. Try things like:

- “Add a Bhutan NDI age check to my Express app before the checkout route.”
- “Build me a ‘Sign in with Bhutan ID’ flow with JWT sessions after the proof.”
- “My proof request is returning 401 from `staging.bhutanndi.com/verifier/v1/...` — what’s wrong?”
- “Generate a verifier that checks Foundational ID + a current Work Permit in one round-trip.”
- “How do I subscribe to `wss://natsdemoclient.bhutanndi.com` and dedup the messages?”

The skill uses **progressive disclosure** — the agent only loads the SKILL.md overview when triggered, and pulls in detailed references (`api.md`, `cookbook.md`, `transport.md`, `credentials.md`, `age-verification.md`, `schema.md`) only when a specific task needs them. That keeps the agent’s context window lean while still giving it complete coverage of the protocol.

Authored by **Sarang Parikh** <arangparikh22@gmail.com>. Corrections welcome — the NDI surface evolves and this doc needs to keep up.